sd22. Fault-Tolerant Large File Upload (chunk / resume)

> First-hand 1p3a P0 (bank #13): "大文件上传如何增加容错" — asked as a single design question. Client-side mirror of sd17's write path; S3 multipart is the reference API. JD tie-in: SDK/client-library behavior (retry, resume, integrity) is literally the JD's "SDK" surface.

Index

1. Crux & why hard

- Naive: one 10 GB POST → any network blip at 99% = restart from zero; no integrity check until the end; server buffers a whale in memory; double-submit uploads it twice.
- Minimum primitives: **chunking** (retry unit), **per-chunk checksum** (catch corruption early), **resumability** (server tells you what it has), **idempotent chunk PUT** (safe retries), **atomic complete** (all-or-nothing visibility) + **whole-file hash** (end-to-end integrity).
- 🗣 "The five words that answer this question: chunk, checksum, resume, idempotent, commit."

2. Clarifying questions (assumed answers)

- File size / network? → up to 10 GB over flaky consumer links
- Concurrent chunk uploads? → yes, 4-8 parallel
- Where do bytes land? → object storage (sd17); this design is the protocol + client
- Dedup / encryption? → mention, out of scope

3. FR / NFR

Req	Target	Enforced by
Survive disconnects	resume, re-send only missing	upload session + ListParts
Corruption caught	per-chunk + whole file	md5/sha per part, final etag check
Retry-safe	same chunk twice = one copy	idempotent PUT by (upload_id, part_no)
Visibility	object appears only complete	commit/complete step
Throughput	parallel parts	independent chunk uploads

4. Back-of-envelope

- 10 GB ÷ 8 MB parts = 1,250 parts; at 1% part-failure rate ≈ 12 retries of 8 MB instead of one 10 GB restart — so what: chunk size = retry cost vs request-overhead balance (8–64 MB sweet spot)
- 4 parallel × 25 MB/s ≈ 100 MB/s → 10 GB in ~2 min on a good link — so what: parallelism is why multipart is also *faster*, not just safer

5. Protocol / API (S3-multipart shaped)

```
POST   /uploads {key, size, sha256}          -> {upload_id, part_size}
PUT    /uploads/{id}/parts/{n}  (Content-MD5) -> {etag_n}     [idempotent]
GET    /uploads/{id}/parts                    -> [{n, etag, size}...]   # resume!
POST   /uploads/{id}/complete {parts:[{n, etag}...]}
       -> 200 {key, etag} | 409 mismatch
DELETE /uploads/{id}                          # abort -> GC parts
```

- Resume = client (or a new client on another machine) calls ListParts and uploads only the gaps.

6. HLD & state machine

```
client ──init──► upload service ──creates──► session (upload_id)
  │ parallel PUT parts (checksummed, idempotent)      [hot path]
  ▼
staging in object store: {upload_id}/{part_no}
  │ complete(parts manifest)
  ▼
verify etags + assemble/commit manifest (sd17 §6)  → object COMMITTED
[async] reaper: abort/GC sessions idle > 24 h

session: INIT ──parts uploading──► PARTIAL ──complete──► COMMITTED
                                      │ abort/expire
                                      ▼ ABORTED (parts GC'd)
```

7. Client upload loop (key code)

```
import hashlib
from concurrent.futures import ThreadPoolExecutor

def upload(api, path, part_mb=8, workers=4, max_retry=5):
    size, file_sha = stat_and_sha256(path)
    up = api.init(path, size, file_sha)                    # or reuse saved upload_id
    have = {p["n"]: p["etag"] for p in api.list_parts(up["id"])}   # resume point

    def send(n):
        block = read_part(path, n, part_mb)
        md5 = hashlib.md5(block).hexdigest()
        for attempt in range(max_retry):
            try:
                return n, api.put_part(up["id"], n, block, md5)     # server verifies md5
            except RetryableError:
                backoff_sleep(attempt)                              # exp + jitter
        raise UploadFailed(n)

    todo = [n for n in range(num_parts(size, part_mb)) if n not in have]
    with ThreadPoolExecutor(workers) as ex:
        for n, etag in ex.map(send, todo):
            have[n] = etag
    return api.complete(up["id"], sorted(have.items()))    # server re-verifies set + file hash
# crash anywhere -> rerun: list_parts returns what landed; only gaps re-sent
```

```
CompletedUpload upload(Api api, Path file, int partMb, int workers) throws Exception {
    Init up = api.init(file);                                   // idempotent by (key, sha)
    Map<Integer, String> have = api.listParts(up.id());         // resume
    ExecutorService pool = Executors.newFixedThreadPool(workers);
    List<Future<Part>> fs = new ArrayList<>();
    for (int n = 0; n < up.partCount(); n++) {
        if (have.containsKey(n)) continue;
        final int part = n;
        fs.add(pool.submit(() -> putWithRetry(api, up.id(), part, readPart(file, part, partMb))));
    }
    for (Future<Part> f : fs) { Part p = f.get(); have.put(p.n(), p.etag()); }
    pool.shutdown();
    return api.complete(up.id(), have);                         // atomic commit
}
// putWithRetry: bounded attempts, exponential backoff + jitter, Content-MD5 each try
```

8. Server: idempotent chunk PUT + complete

- PUT part: verify Content-MD5 against received bytes → store at `{upload_id}/{part_no}` (overwrite-same = harmless) → upsert row, return etag. Same part twice → same etag, no duplicate bytes.

- Complete: verify client's part manifest matches stored (count, etags, contiguous numbering, sizes) → verify assembled hash = declared sha256 → sd17 manifest commit → parts become the object's chunks (no copy needed if part size = chunk size!)
- Complete is idempotent too: re-complete of a COMMITTED session returns the same 200.

9. Data model

```
CREATE TABLE uploads (
  upload_id  TEXT PRIMARY KEY,
  bucket     TEXT NOT NULL, key TEXT NOT NULL,
  declared_size BIGINT NOT NULL,
  declared_sha  TEXT NOT NULL,
  part_size  INT NOT NULL,
  state      TEXT NOT NULL DEFAULT 'INIT',   -- INIT|PARTIAL|COMMITTED|ABORTED
  created_at TIMESTAMPTZ NOT NULL,
  expires_at TIMESTAMPTZ NOT NULL            -- reaper GC deadline
);
CREATE TABLE upload_parts (
  upload_id TEXT REFERENCES uploads,
  part_no   INT NOT NULL,
  etag      TEXT NOT NULL,                   -- md5 of part
  size_b    INT NOT NULL,
  PRIMARY KEY (upload_id, part_no)           -- upsert => idempotent PUT
);
```

10. Failure modes — the 容错 checklist (say all five)

1. Network blip mid-part → bounded retries with backoff+jitter; only that part re-sent
2. Client crash → resume: init returns existing session (idempotent by key+sha), ListParts fills `have`
3. Corruption in flight → per-part Content-MD5 rejected at write time; final sha256 catches assembly/ordering bugs
4. Duplicate submit (double-click, retried complete) → idempotent init/PUT/complete — no double object, no double bytes
5. Abandoned uploads → expires_at reaper aborts + GCs parts (storage doesn't leak)

11. Trade-offs & defeaters

- Defeater: "整文件重传" — the whole point is chunk-level retry; never restart from zero
- Defeater: "先合并再校验" — verify per part on receipt; the final hash is a second seal, not the first check
- Defeater: "server holds parts in memory / temp disk then copies" — stage parts *as* future chunks in the object store; complete = metadata commit, zero byte copy
- Part size: small = more requests + metadata; big = expensive retries; 8–64 MB, adaptive on link quality

12. Pop-ups

- ❓ Browser client? → same protocol; presigned per-part URLs so bytes skip your API tier (sd17 §5 pattern)
- ❓ Pause across devices? → session state is server-side; any client with upload_id + the file can resume
- ❓ Encryption? → TLS in flight; at-rest via storage layer; client-side envelope encryption if end-to-end needed
- **L5:** delta/dedup — content-defined chunking + cid lookup skips parts the server already has (backup-tool territory)

13. Memorization card

- 🗣️ chunk → checksum → resume → idempotent → commit (+ final whole-file hash)
- API: init / PUT part (Content-MD5, upsert) / ListParts / complete (verify + atomic) / abort + reaper
- Parallel parts = faster AND safer; part = retry unit = storage chunk (zero-copy complete)
- Idempotency everywhere: init by (key, sha), part by (upload_id, n), complete re-entrant